

Modelos Neurais: Utilizando Pipelines na Plataforma Hugging Face

Aula 21 – Introdução ao Processamento de Linguagem Natural (PLN)

Eduardo Corrêa Gonçalves

15/06/2026

Sumário

Introdução

O que é Hugging Face ?

O que é Arquitetura Transformer ?

Casos Práticos com o método Pipeline

Conceito: O que é Pipeline ?

Caso 1: Análise de sentimentos com Transformers (modelo default)

Conceito: *card* do modelo

Caso 2: Análise de sentimentos com Transformers (escolhendo o modelo)

Caso 3: Detecção de Frases Neutras

Caso 4: Trabalhando com um modelo multilingual

Caso 5: Classificação zero-shot

Conceito: Classificação zero-shot *versus* one-shot *versus* few-shot

Caso 6: Sumarização

Caso 7: Reconhecimento de Entidades Nomeadas

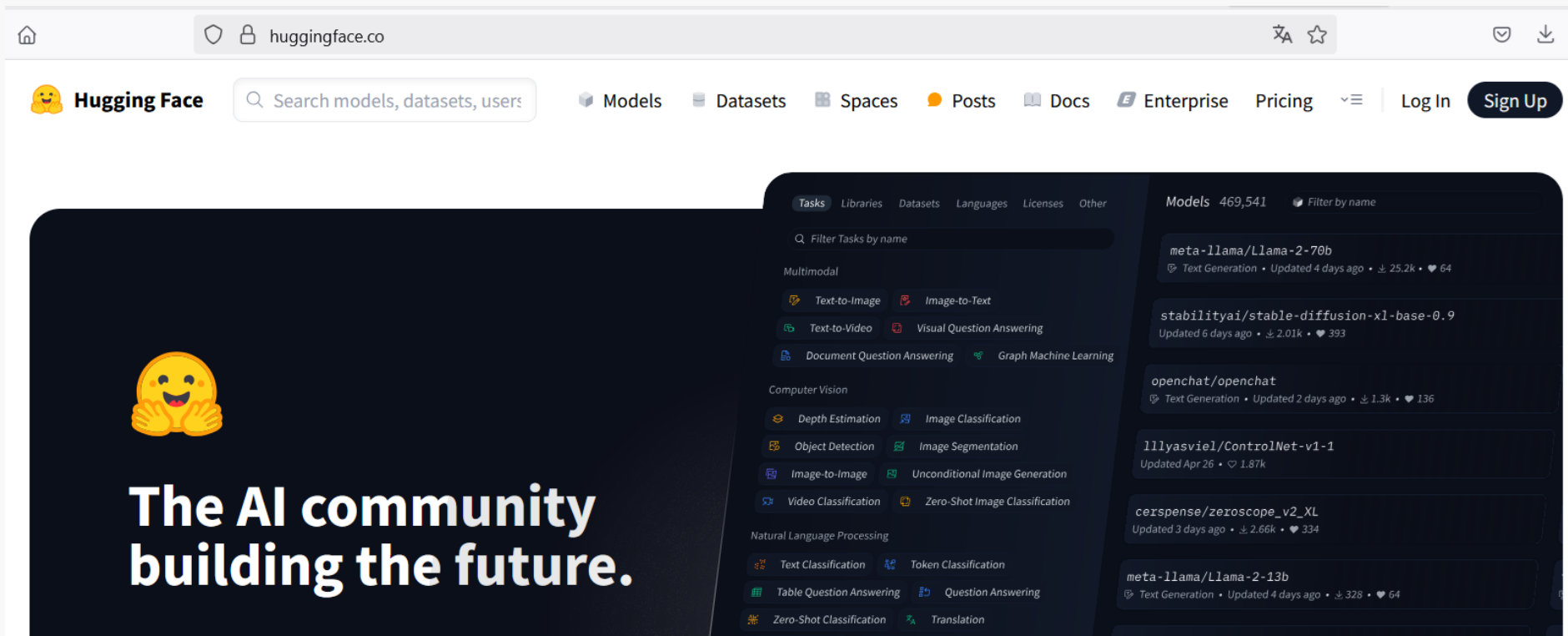
Introdução (1/7)

- O que é Hugging Face ?
 - Hugging Face (<https://huggingface.co/>) é, atualmente, a mais importante e conhecida **plataforma colaborativa de aprendizado de máquina**.
 - Nesta plataforma, usuários de todo o mundo compartilham:
 - modelos de aprendizado de máquina;
 - bibliotecas de embeddings treinados;
 - modelos de linguagem treinados;
 - bases de dados;
 - notebooks Python;
 - *e muitas outras coisas não apenas de PLN... Mas também para IA com imagens, áudios, vídeos etc.*
 - É uma espécie de Github voltado para IA



Introdução (2/7)

- Na comunidade de PLN, a plataforma se tornou famosa por 2 motivos principais:
 - Facilitou a utilização do modelo **Transformers** (estado da arte em modelos de linguagem) com a **biblioteca transformers**.
 - Facilitou o compartilhamento de modelos de **IA Generativa de código aberto** de última geração, como Llama, Gemma e outros.



Introdução (3/7)

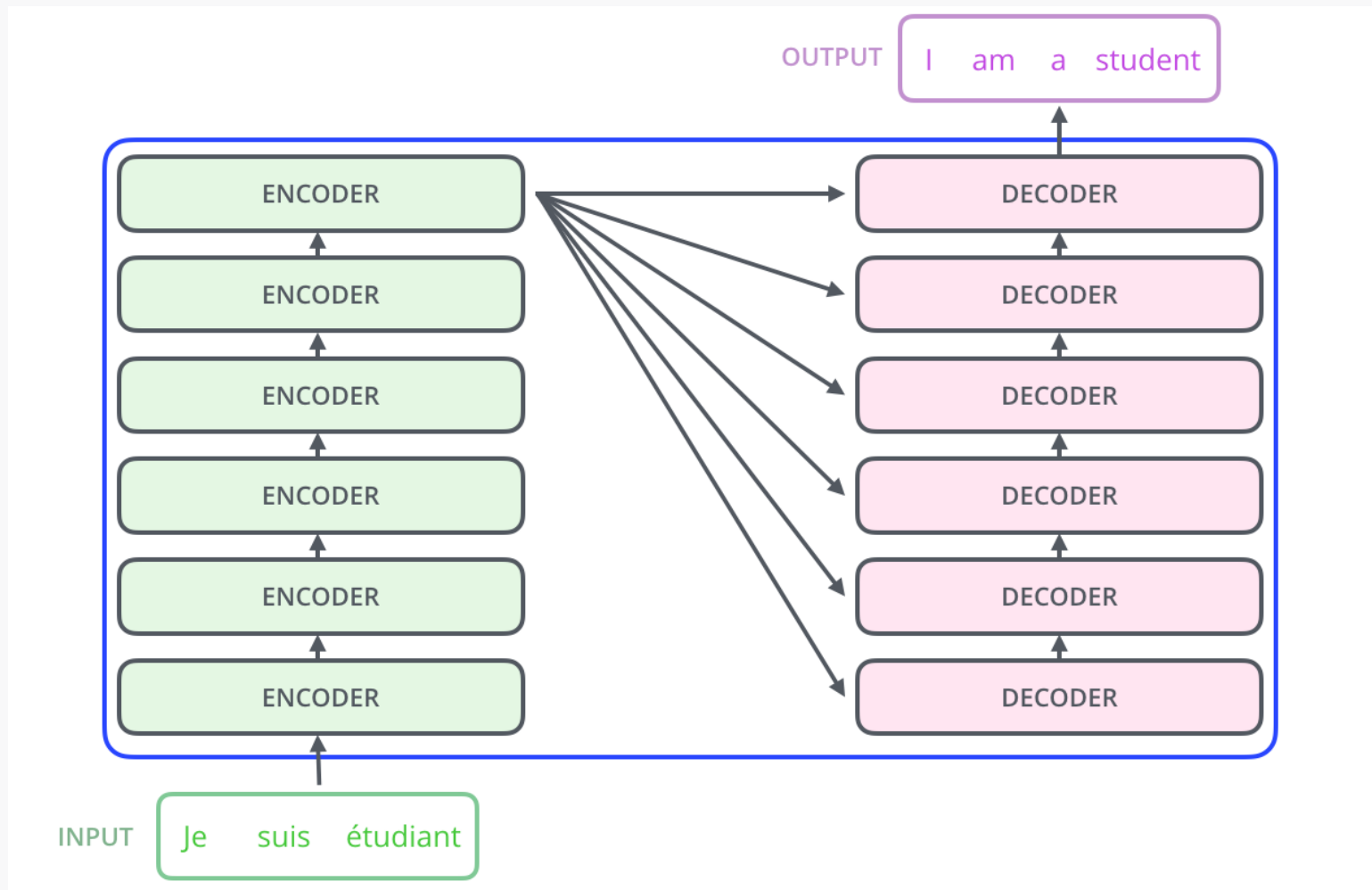
- Mas o que é Transformer ?
 - É a arquitetura mais avançada e que se tornou a padrão para a construção de modelos de linguagem (Jurafsky e Martin, 2025).
 - O Transformer é uma rede neural com uma estrutura específica e bastante complexa, que inclui um mecanismo inovador chamado de atenção (*self-attention*).
 - Olhando como uma caixa-preta, o Transformer é um modelo que recebe texto como entrada e produz texto como saída.



Fonte: Alammar (2018)

Introdução (4/7)

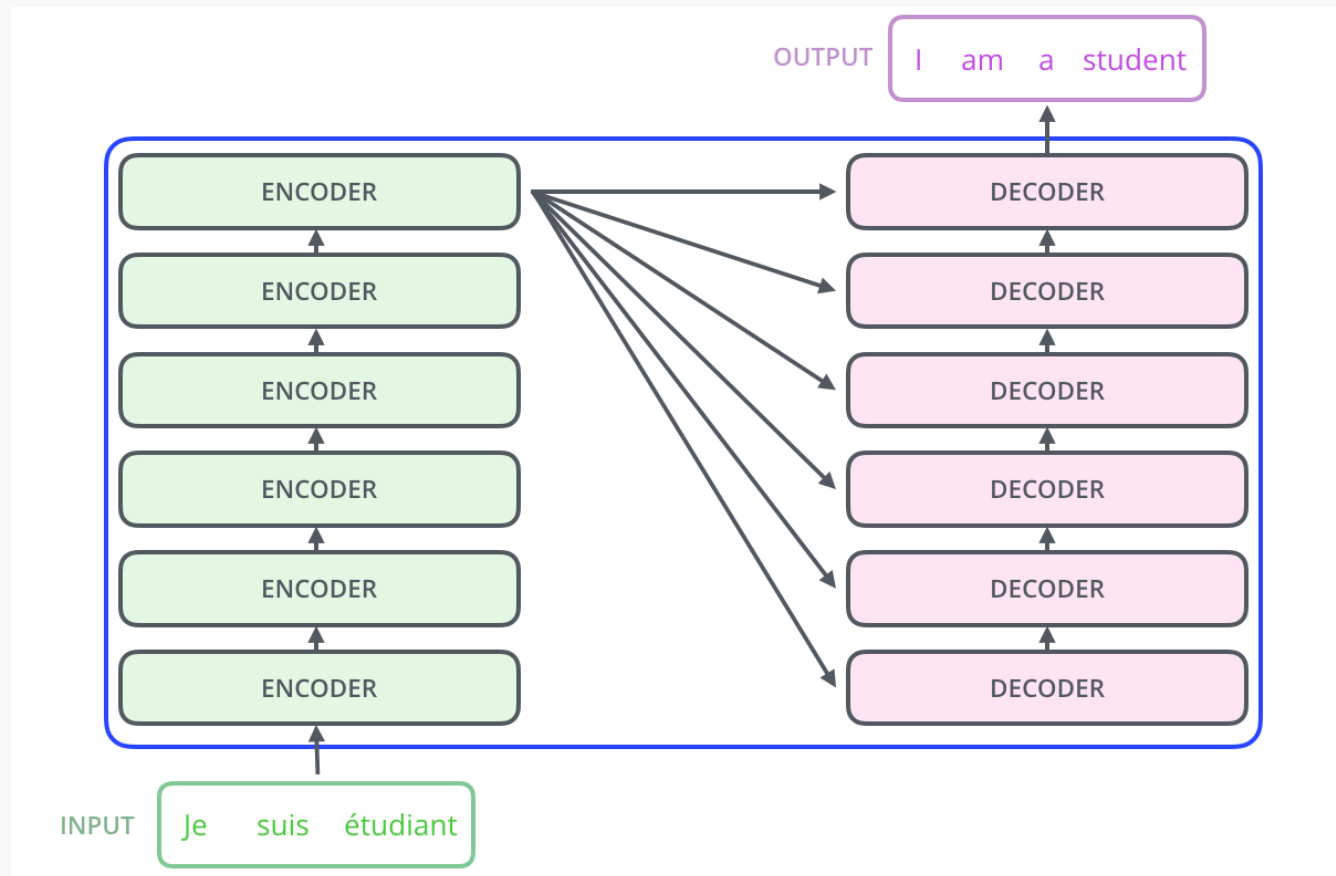
- Mas o que é Transformer ?
 - Se “abrirmos” a caixa-preta, veremos que um Transformer é composto por 2 componentes: **encoders** e **decoders** (são redes neurais empilhadas)



Fonte: Alammr (2018)

Introdução (5/7)

- Mas o que é Transformer ?
 - **Encoders**: recebem texto como entrada e tem por objetivo gerar a melhor representação vetorial (*embeddings*) possível para o texto na saída.
 - **Decoders**: recebem uma representação vetorial do texto como entrada e têm por objetivo gerar o melhor texto possível na saída.

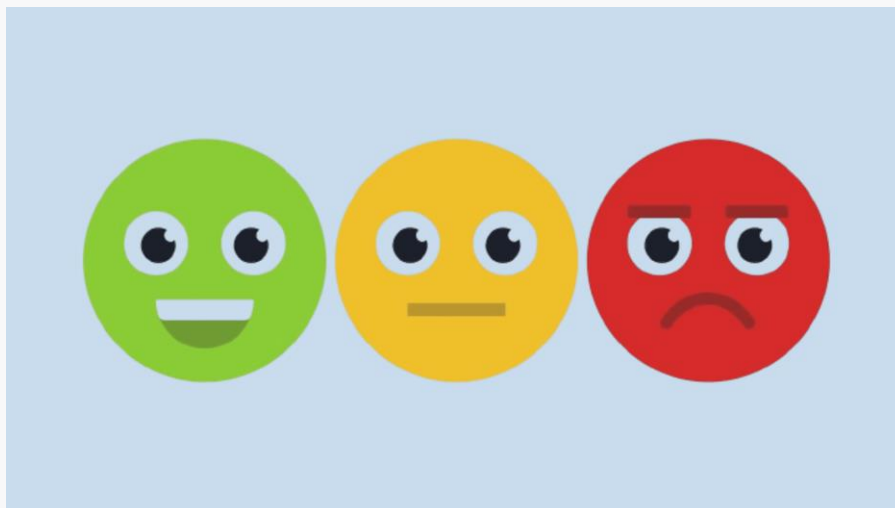


Introdução (6/7)

- Mas o que é Transformer ?
 - Alguns modelos baseados no Transformer são chamados de **encoder-only** porque utilizam só o componente encoder da arquitetura.
 - É o caso do BERT e derivados.
 - São modelos que estão preocupados apenas em gerar o melhor embeddings possível para um texto.
 - Outros são **decoder-only** porque utilizam só o componente decoder.
 - É o caso de LLMs como GPT, Llama e Gemma.
 - Converter o texto de entrada usando embeddings comuns (Word2vec, por exemplo) e concentram seus esforços em produzir ótimos textos de saída.
 - O espaço que seria ocupado pelos encoders fica livre e, assim, a quantidade de parâmetros dos decoders pode crescer.

Introdução (7/7)

- **Aula de hoje**
 - Para uma visão geral sobre os recursos do Hugging Face, consulte o artigo: <https://www.alura.com.br/artigos/hugging-face>.
 - Para uma visão em alto nível da arquitetura Transformer, um bom ponto de partida é o artigo de Alammr (2018).
 - Na aula de hoje, mostraremos um exemplo de utilização prática da **biblioteca transformers** modelos pré-treinados da Hugging Face através do modo **Pipeline**.



Fonte: Carrascosa (2024)

Pipelines e biblioteca transformer (1/3)

- **Pipelines do Hugging Face**

- Há duas formas de utilizar os modelos do Hugging Face: **Pipelines** e **Auto Classes**.
- O modo **Pipeline** exige **menos código** e oferece um **nível de abstração alto**, sendo por isso bom para um primeiro contato com a plataforma.
- No modo pipeline você consegue realizar tarefas (classificação, por exemplo), com pouquíssimas linhas de código.
 - Não precisa tokenizar, extrair embeddings ou treinar um modelo explicitamente!!!
- Muito fácil de usar, facilita demais lidar com a biblioteca **transformer**. Porém, tende a ser menos eficaz e eficiente.
- Vamos apresentar um primeiro exemplo com **análise de sentimentos** (base **sentiment140**) e comparar os resultados com os de todos os métodos vistos em nosso curso.

Pipelines e biblioteca transformer (2/3)

- **Habilitando GPUs**

- Para trabalhar com a **biblioteca transformer** no Colab, indica-se habilitar o uso de GPUs para permitir a paralelização da execução do código.
- Vá ao menu “Ambiente de Execução,” escolha “Alterar o tipo de ambiente de execução”, selecione **GPUs:T4** e depois clique em **Salvar**.

The screenshot shows the Google Colab interface for a notebook titled 'Hugging Face Transformers - Pipelines.ipynb'. The 'Ambiente de execução' (Runtime) menu is open, displaying various execution options. The 'Alterar o tipo de ambiente de execução' (Change runtime type) option is highlighted. A dialog box titled 'Alterar o tipo de ambiente de execução' is displayed, showing the current runtime type as 'Python 3'. Under the 'Acelerador de hardware' (Hardware accelerator) section, the 'GPUs: T4' option is selected with a radio button. Other options include CPU, GPUs: A100, GPUs: L4, TPU v2-8, TPU v6e-1, and TPU v5e-1. At the bottom of the dialog, there is a link to 'Compre mais unidades de computação' (Buy more computing units) for premium GPU access. The 'Salvar' (Save) button is visible at the bottom right of the dialog.

Hugging Face Transformers - Pipelines.ipynb

Arquivo Editar Ver Inserir Ambiente de execução Ferramentas Ajuda

ando + Código + Texto

Referências:

- Aula 18 do curso de PLN
- [Hugging face: o que é e c](#)
- [Hugging face pipeline: A](#)
- [Simple NLP Pipelines wit](#)
- [Hugging Face Transform](#)
- [The Illustrated Transform](#)

▼ PASSO 0: importar a bib

- Demora cerca de 40s

[] from transformers im

Executar tudo Ctrl+F9

Executar antes Ctrl+F8

Executar a célula em foco Ctrl+Enter

Executar seleção

Executar célula e abaixo

Interromper execução

Reiniciar sessão

Reiniciar sessão e executar tudo

Desconectar e excluir ambiente de execução

Alterar o tipo de ambiente de execução

Gerenciar sessões

Ver recursos

Alterar o tipo de ambiente de execução

Tipo de ambiente de execução

Python 3

Acelerador de hardware ?

☐ CPU ☒ GPUs: T4 ☐ GPUs: A100 ☐ GPUs: L4

☐ TPU v2-8 ☐ TPU v6e-1 ☐ TPU v5e-1

Quer acesso a GPUs premium? [Compre mais unidades de computação](#)

Cancelar Salvar

Pipelines e biblioteca transformer (3/3)

- **Importando a biblioteca**
 - Feito isso, vamos começar importando a biblioteca transformers (que já vem instalada no Colab)
 - Em seguida apresentaremos diferentes casos práticos utilizando essa biblioteca.

```
from transformers import pipeline
```

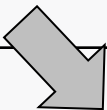
- Vamos agora ver alguns exemplos compartilhados no notebook abaixo:

<https://colab.research.google.com/drive/1Q7oe-vT3vjB6BGBHZsA-xolA5q3GFqaj?usp=sharing>

Caso 1: Análise de Sentimentos (padrão) (1/5)

- Pipeline para classificação de sentimentos com o modelo default
 - Esse primeiro exemplo mostra o uso mais básico do pipeline para classificação de sentimentos
 - Por padrão, o modelo default é `distilbert-base-uncased-finetuned-sst-2-english` (aparentemente só serve para Inglês)
 - **Utilização simples:** basta instanciar o classificador via pipeline e depois obter as predições passando um array com as frases

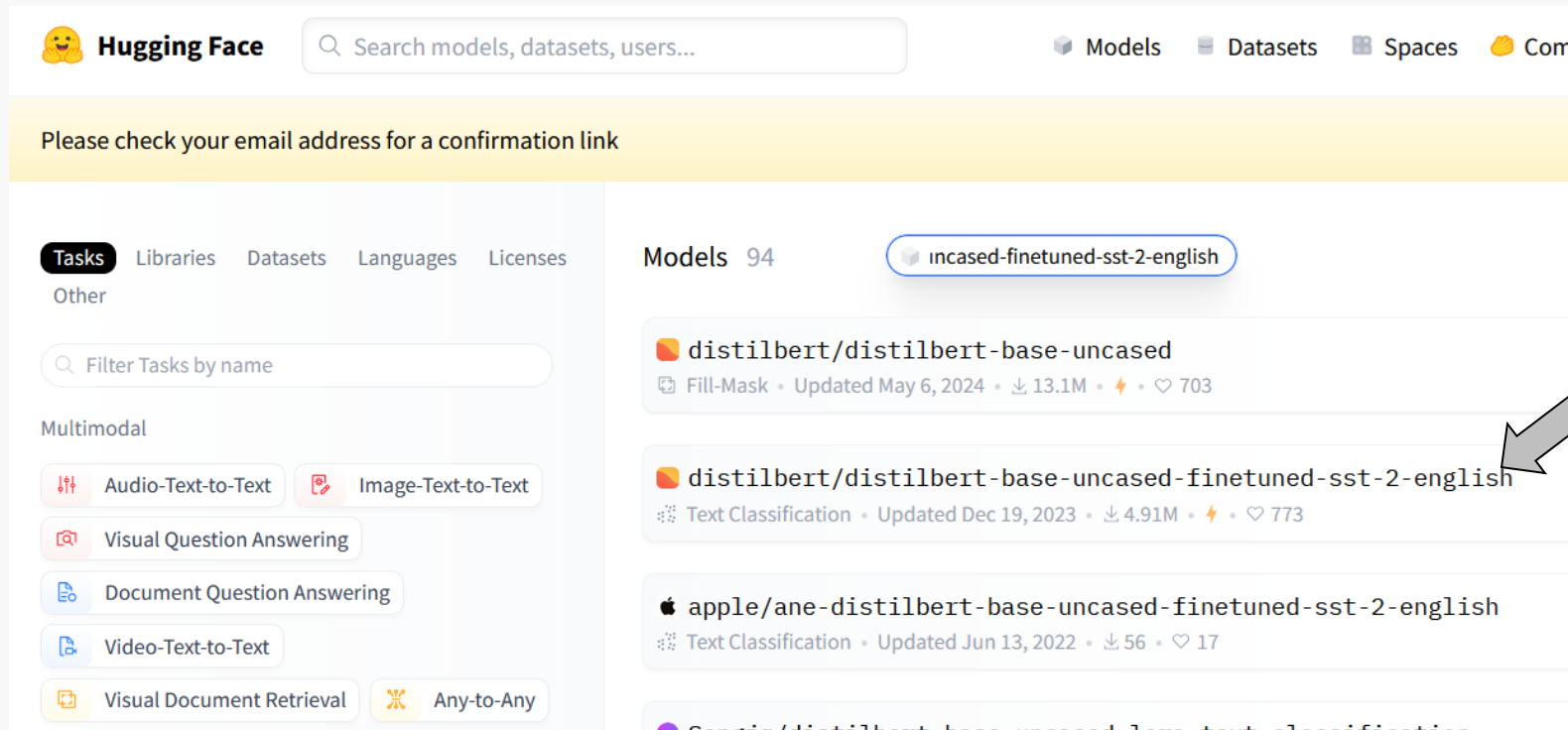
```
# Exemplo básico do Pipeline para classificação de sentimentos
classificador = pipeline("sentiment-analysis")
frases = ['@stellargirl I loooooooooovvvvvveee my Kindle2.',
          'Blah, blah, blah same old same old. ']
preds = classificador(frases)
print(preds)
```



```
No model was supplied, defaulted to distilbert/distilbert-base-uncased-finetuned-sst-2-english and revisio
Using a pipeline without specifying a model name and revision in production is not recommended.
Device set to use cuda:0
[{'label': 'POSITIVE', 'score': 0.9696942567825317}, {'label': 'NEGATIVE', 'score': 0.9991746544837952}]
```

Caso 1: Análise de Sentimentos (padrão) (2/5)

- **Card do Modelo** – você pode visualizar as características de qualquer modelo do hugging face, acessando o *card* do modelo em questão.
 - Por exemplo, para acessar o cardo do `distilbert-base-uncased-finetuned-sst-2-english`:
 - Acesse <https://huggingface.co/>
 - Clique em **Models** (menu superior) e busque por `distilbert-base-uncased-finetuned-sst-2-english`



apareceu em
2º na busca...

cuidado, pois
os resultados
por padrão
são
ordenados por
“popularidade”

Caso 1: Análise de Sentimentos (padrão) (3/5)

- Ao clicar no nome do modelo, você verá o seu card
 - Apresenta dados sobre o tamanho do modelo
 - Processo de treinamento
 - Exemplo de código (nem sempre no modo Pipeline)
 - Limitações
 - ...

The screenshot shows the Hugging Face model card for 'DistilBERT base uncased finetuned SST-2'. It includes a 'Table of Contents' with links to Model Details, How to Get Started With the Model, Uses, Risks, Limitations and Biases, and Training. The 'Model Details' section provides a description of the model as a fine-tune checkpoint of DistilBERT-base-uncased, fine-tuned on SST-2, with an accuracy of 91.3 on the dev set. It also lists key information: Developed by: Hugging Face, Model Type: Text Classification, Language(s): English, License: Apache-2.0, Parent Model: DistilBERT, and Resources for more information: Model Documentation and DistilBERT paper. The 'How to Get Started With the Model' section includes an example of single-label classification with Python code.

DistilBERT base uncased finetuned SST-2

Table of Contents

- [Model Details](#)
- [How to Get Started With the Model](#)
- [Uses](#)
- [Risks, Limitations and Biases](#)
- [Training](#)

Model Details

Model Description: This model is a fine-tune checkpoint of [DistilBERT-base-uncased](#), fine-tuned on SST-2. This model reaches an accuracy of 91.3 on the dev set (for comparison, Bert bert-base-uncased version reaches an accuracy of 92.7).

- **Developed by:** Hugging Face
- **Model Type:** Text Classification
- **Language(s):** English
- **License:** Apache-2.0
- **Parent Model:** For more details about DistilBERT, we encourage users to check out [this model card](#).
- **Resources for more information:**
 - [Model Documentation](#)
 - [DistilBERT paper](#)

How to Get Started With the Model

Example of single-label classification:

```
import torch
from transformers import DistilBertTokenizer, DistilBertForSequenceClassification

tokenizer = DistilBertTokenizer.from_pretrained("distilbert-base-uncased-finetune-sst2")
model = DistilBertForSequenceClassification.from_pretrained("distilbert-base-uncased-finetune-sst2")
```

Caso 1: Análise de Sentimentos (padrão) (4/5)

- Classificando a base **sentiment140**
 - [Detalhes no notebook](#)
 - O resultado final não foi grandes coisas...

Matriz de Confusão:

[[47 8]

[14 50]]

Métricas de Desempenho Preditivo:

Acurácia: 0.815

Precisão: 0.862

Revocação: 0.781

F1-Score: 0.820

Caso 1: Análise de Sentimentos (padrão) (5/5)

- Comparação com outros métodos

Método	Feature	Acurácia	Precisão	Recall	F1
Naïve Bayes	BoW (1734 atributos)	0,840	0,857	0,844	0,850
Regressão Logística	Eng. Características (6 atributos)	0,773	0,761	0,844	0,800
Rede Neural FeedForward	Eng. Características (6 atributos)	0,773	0,785	0,797	0,791
Rede Neural FeedForward	TF-IDF (1206 atributos)	0,815	0,850	0,797	0,823
Rede Neural FeedForward	Embeddings spaCy (300 atributos)	0,840	0,817	0,906	0,859
Transformer distilbert-base-uncased-finetuned-sst-2-english	-	0,815	0,862	0,781	0,820

Caso 2: Análise Sentim. (escolhendo o modelo) (1/2)

- Pipeline para classificação de sentimentos com o modelo default
 - Com o parâmetro **model** podemos escolher outro modelo para o Pipeline
 - Escolhi o modelo **bertweet-base-sentiment-analysis**, que é bem conhecido e foi treinado especificamente para análise de sentimentos com textos do Twitter.
 - **Obs.:** nem todos os (milhares) de modelos disponíveis na HuggingFace funcionam no modo Pipeline. (poucos foram habilitados...)

Pipeline para classificação de sentimentos com escolha de um modelo

```
modelo2 = pipeline('sentiment-analysis', model = 'finiteautomata/bertweet-base-sentiment-analysis')
```

```
config.json: 100% ██████████ 949/949 [00:00<00:00, 10.9kB/s]
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance,
WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular H
pytorch_model.bin: 100% ██████████ 540M/540M [00:06<00:00, 121MB/s]
Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular HTTP download. For better performance,
WARNING:huggingface_hub.file_download:Xet Storage is enabled for this repo, but the 'hf_xet' package is not installed. Falling back to regular H
model.safetensors: 100% ██████████ 540M/540M [00:06<00:00, 72.0MB/s]
tokenizer_config.json: 100% ██████████ 338/338 [00:00<00:00, 23.5kB/s]
vocab.txt: 100% ██████████ 843k/843k [00:00<00:00, 5.73MB/s]
bpe.codes: 100% ██████████ 1.08M/1.08M [00:00<00:00, 8.34MB/s]
added_tokens.json: 100% ██████████ 22.0/22.0 [00:00<00:00, 1.07kB/s]
special_tokens_map.json: 100% ██████████ 167/167 [00:00<00:00, 10.8kB/s]
emoji is not installed, thus not converting emoticons or emojis into text. Install emoji: pip3 install emoji==0.6.0
```

Caso 2: Análise Sentim. (escolhendo o modelo) (2/2)

- Comparação com outros métodos

Método	Feature	Acurácia	Precisão	Recall	F1
Naïve Bayes	BoW (1734 atributos)	0,840	0,857	0,844	0,850
Regressão Logística	Eng. Características (6 atributos)	0,773	0,761	0,844	0,800
Rede Neural FeedForward	Eng. Características (6 atributos)	0,773	0,785	0,797	0,791
Rede Neural FeedForward	TF-IDF (1206 atributos)	0,815	0,850	0,797	0,823
Rede Neural FeedForward	Embeddings spaCy (300 atributos)	0,840	0,817	0,906	0,859
Transformer distilbert-base-uncased- finetuned-sst-2-english	-	0,815	0,862	0,781	0,820
Transformer bertweet-base- sentiment-analysis	-	0,924	0,887	0,984	0,933

Caso 3: Frases Neutras (1/2)

- Card do **bertweet-base-sentiment-analysis**
 - De acordo com o card, ele consegue classificar frases neutras
 - Os labels são POS, NEG e NEU
 - O modelo foi treinado apenas com tweets em Inglês.

bertweet-sentiment-analysis

Repository: <https://github.com/finiteautomata/pysentimiento/>

Model trained with SemEval 2017 corpus (around ~40k tweets). Base model is BERTweet, a RoBERTa model trained on English tweets.

Uses POS, NEG, NEU labels.

License

pysentimiento is an open-source library for non-commercial use and scientific research purposes only. Please be aware that models are trained with third-party datasets and are subject to their respective licenses.

1. [TASS Dataset license](#)
2. [SEMEval 2017 Dataset license](#)

Caso 3: Frases Neutras (2/2)

- **Classificando frases neutras**
 - No teste abaixo, não parece ter se saído tão bem...

```
# Pipeline para classificação de sentimentos com escolha de um modelo
modelo2 = pipeline('sentiment-analysis', model = 'finiteautomata/bertweet-
base-sentiment-analysis')
```

```
modelo2(["I don't know if I either enjoyed or not",
        "I don't know if I either enjoyed or not, but I probably did",
        "I don't know if I either enjoyed or not, perhaps I did",
        "I don't know if I either enjoyed or not, completely unsure"])
```

```
[{'label': 'NEG', 'score': 0.5803262591362},
 {'label': 'POS', 'score': 0.7210913300514221},
 {'label': 'NEU', 'score': 0.8258950710296631},
 {'label': 'NEG', 'score': 0.7246370315551758}]
```

Caso 4: Modelo Multilingual (1/2)

- O que é modelo multilingual ?
 - Como o nome diz, é um modelo treinado com textos de vários idiomas.
 - Se Português for uma delas, podemos usar modelo para textos em nosso idioma !
 - **Ex.:** [cardiffnlp/twitter-xlm-roberta-large-2022](#) é um modelo deste tipo, para análise de sentimentos, treinado com 1 bilhão de tweets.

The screenshot shows the Hugging Face interface for the model `cardiffnlp/twitter-xlm-roberta-large-2022`. At the top, there are tags for `Transformers`, `PyTorch`, `multilingual`, and `License: mit`. Below the tags, there are tabs for `Model card`, `Files and versions`, and `Community`. The `Model card` tab is selected, showing the following text:

Twitter-XLM-Roberta-large

This is a XLM-T large language model specialised on Twitter. The base model is a multilingual XLM-R which was re-trained on over 1 billion tweets from different languages until December 2022.

To evaluate this and other LMs on Twitter-specific data, please refer to the [XLM-T main repository](#). A base-size XLM-T model and sample code is available [here](#).

Finally, this model is fully compatible with the [TweetNLP library](#).

Caso 4: Modelo Multilingual (2/2)

- **Classificando frases em Português**
 - O modelo é pesado e demora bastante para ser carregado.
 - Porém, no meu pequeno teste classificou 5 frases em Português corretamente. *Observe também as probabilidades atribuídas*

```
# Carregando o modelo (demora)
```

```
pipe_pt = pipeline("text-classification", model="cardiffnlp/twitter-xlm-roberta-base-sentiment")
```

```
frases = ["lixo total",  
          "imperdível esse show, curti demais",  
          "uma porcaria muito grande e repugnante",  
          "quase gostei, mas não foi o caso",  
          "valeu a pena o dinheiro investido"]
```

```
pipe_pt (frases)
```

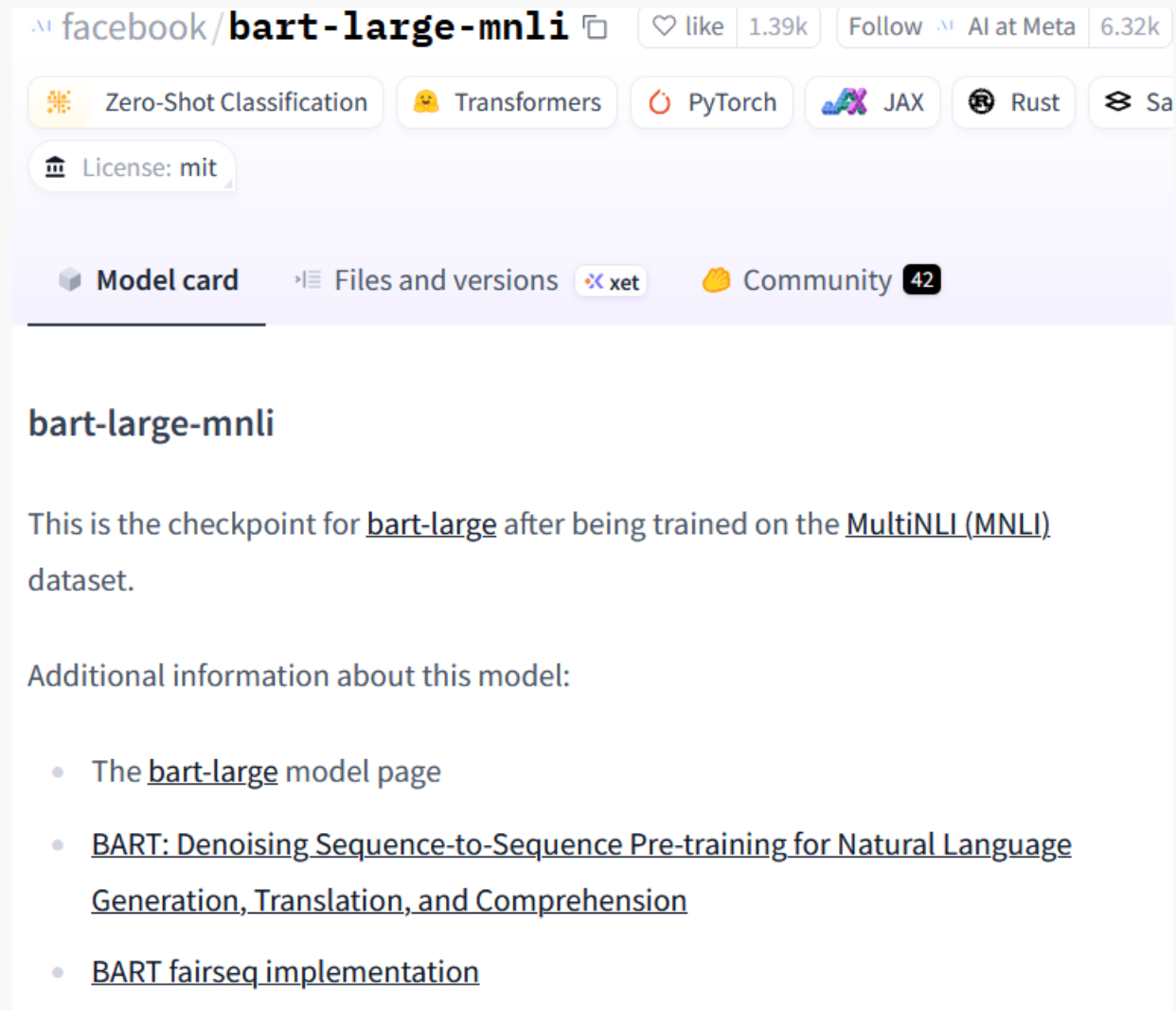
```
[{'label': 'negative', 'score': 0.9413319230079651},  
{ 'label': 'positive', 'score': 0.9341555833816528},  
{ 'label': 'negative', 'score': 0.9627209305763245},  
{ 'label': 'negative', 'score': 0.6110736727714539},  
{ 'label': 'positive', 'score': 0.7049989700317383}]
```

Caso 5: Classificação Zero-shot (1/5)

- O que é classificação zero-shot ?
 - É uma técnica que permite com que modelos de linguagem classifiquem textos em categorias nunca antes vistas, tendo por base apenas as descrições das categorias.
 - Basicamente o modelo recebe apenas uma frase e um *prompt* que solicita com que a frase seja classificada em um conjunto de possíveis categorias.
 - Pode ser **qualquer frase** e um conjunto de **categorias qualquer !!!!!**
 - Para mais informações consulte:
<https://joeddav.github.io/blog/2020/05/29/ZSL.html>
 - No hugging face, o modelo com mais downloads é **facebook/bart-large-mnli**

Caso 5: Classificação Zero-shot (2/5)

- Card do **facebook/bart-large-mnli**
 - Usar o modelo é fácil... Mas cada um é baseado em conceitos e princípios diferentes (o ideal é conhecer esses conceitos antes de usar o modelo)



The screenshot shows the Hugging Face Model Card for **facebook/bart-large-mnli**. At the top, it displays the repository name, a copy icon, and engagement metrics: 1.39k likes and 6.32k follows. Below this, a row of tags includes Zero-Shot Classification, Transformers, PyTorch, JAX, Rust, and SafeTensors. The license is specified as MIT. A navigation bar at the bottom of the header includes links for Model card, Files and versions, xet, and Community (42 members).

bart-large-mnli

This is the checkpoint for **bart-large** after being trained on the **MultiNLI (MNLI)** dataset.

Additional information about this model:

- The [bart-large](#) model page
- [BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension](#)
- [BART fairseq implementation](#)

Caso 5: Classificação Zero-shot (3/5)

- **Classificando o gênero de filmes em função do resumo**
 - Dessa vez, o pipeline é “zero-shot-classification”
 - O exemplo abaixo classifica 2 filmes

```
# Carregando o modelo
```

```
classifier = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")
```

```
# resumo no IMDB de 2 filmes distintos: o 1o é ação, o 2o drama/romance
```

```
filmes = ['Arrebentando em NY', 'O Casamento de Maria Braun']
```

```
plots = ["A young man visiting and helping his uncle in New York City finds himself  
forced to fight a street gang and the mob with his martial art skills",
```

```
        "Maria marries Hermann Braun in the last days of World War II, only for him to  
go missing in the war. Alone, Maria puts to use her beauty and ambition in order to  
find prosperity during Germany's economic miracle of the 1950s."]
```

```
candidate_labels = ['action', 'comedy', 'drama', 'horror', 'romance']
```

```
classifier(plots, candidate_labels, multilabel=False)
```

Caso 5: Classificação Zero-shot (4/5)

- **Classificando o gênero de filmes em função do resumo**
 - No primeiro resumo, classificou muito bem
 - No segundo, não tão bem

{'sequence': 'A young man visiting and helping his uncle in New York City finds himself forced to fight a street gang and the mob with his martial art skills',

'labels': ['**action**', 'drama', 'horror', 'comedy', 'romance'], 'scores':
[**0.8328219056129456**, 0.07260971516370773, 0.04249740391969681,
0.033010873943567276, 0.019060121849179268]},

{'sequence': '"Maria marries Hermann Braun in the last days of World War II, only for him to go missing in the war. Alone, Maria puts to use her beauty and ambition in order to find prosperity during Germany's economic miracle of the 1950s."',

'labels': ['**romance**', '**action**', '**drama**', 'comedy', 'horror'], 'scores':
[**0.35294824838638306**, **0.3466242253780365**, **0.20213568210601807**,
0.05161937326192856, 0.046672508120536804]}]

Caso 5: Classificação Zero-shot (5/5)

- Com modelos multilinguais, podemos classificar textos em Português
 - Exemplo: “MoritzLaurer/mDeBERTa-v3-base-mnli-xnli”

```
# Carregando o modelo
classifier = pipeline("zero-shot-classification",
                      model="MoritzLaurer/mDeBERTa-v3-base-mnli-xnli")

frase = "O presidente aumentou o salário mínimo"
classes_candidatas = ["política", "economia", "esportes", "artes"]

classifier(frase, classes_candidatas, multi_label=False)
```

```
{'sequence': 'O presidente aumentou o salário mínimo',
 'labels': ['política', 'economia', 'artes', 'esportes'], 'scores': [0.6227890253067017,
 0.30162930488586426, 0.047616295516490936, 0.027965322136878967]}
```

Caso 6: Classif. One-shot e Few-shot (1/x)

- **Prompt zero-shot**
 - Conforme acabamos de ver, podemos usar o prompt zero-shot para classificar textos no Hugging Face.

Classify the following input text into one of the following three categories:
[positive, negative, neutral]

Input Text: Hugging Face is awesome for making all of these state of the art models available!

Sentiment: positive

Fonte: <https://huggingface.co/tasks/zero-shot-classification>

- Mas além do zero-shot, existem também os prompts:
 - One-shot
 - Few-shot

Classificação One-shot e Few-shot (1/3)

- Prompt one-shot
 - Conforme acabamos de ver, podemos usar o prompt zero-shot para classificar textos no Hugging Face.

Is the following text sexist? Answer yes or no

'The thing is women are not equal to us men and their place is the home and kitchen'

Answer:

GPT-3 response:

Yes.

Fonte: Chiu; Alexander (2021)

- Mas além do zero-shot, existem também os prompts:
 - One-shot
 - Few-shot
 - *Vamos ver o que é isso !!!*

Classificação One-shot e Few-shot (2/3)

- **Prompt one-shot**
 - Nesse caso, primeiro é dado ao modelo **um exemplo** de uma categoria
 - Em seguida, um texto é fornecido e pedimos com que o modelo diga se ele é da mesma categoria do exemplo fornecido.

Prompt:

The following text in quotes is sexist:
'Feminism is a very terrible disease'

Is the following text sexist? Answer yes or no.
'She is heavily relying on him to turn the other cheek. . . tough talking demon infested woman.'

Answer:
GPT-3 response:
Yes

Fonte: Chiu; Alexander (2021)

Classificação One-shot e Few-shot (3/3)

- **Prompt few-shot**

- Nesse caso, ao menos **mais de um exemplo** é fornecido ao modelo .
- Normalmente, pelo menos um exemplo de cada classe
- Em seguida, um texto é fornecido e pedimos com que o modelo diga qual é a classe.

Prompt:

‘Now they know better than this shit lol they dudes. The stronger sex. The man supremacy’: sexist.

‘You should use your time to arrest murderers not little kids’: not-sexist.

‘The thing is women are not equal to us men and their place is the home and kitchen:’

GPT-3 response:

Sexist

Fonte: Chiu; Alexander (2021)

Caso 6: Sumarização (1/2)

- Pipelines não são apenas para classificação !!!
 - Existem pipelines para diversas outras tarefas.
 - No exemplo abaixo, um exemplo da tarefa de sumarização – produção do resumo de uma notícia, limitando em 100 tokens.
 - Foi utilizado o conhecido modelo **t5**, que é do tipo encoder-decoder.

```
import torch
```

```
sumarizador = pipeline(task="summarization", model="google-t5/t5-base",  
                        torch_dtype=torch.float16, device=0)
```

```
entrada = """
```

```
Uma turista que visitava o Cristo Redentor neste domingo (25) foi surpreendida  
por um quati que roubou o seu sanduíche e fugiu. O animal chegou devagarzinho e,  
sorrateiramente, meteu a patinha no lanche da moça e correu. Pessoas que estavam  
ao redor registraram a cena. Como o animal estava pertinho deles, os visitantes  
começaram a filmar e flagraram o animalzinho garantindo a refeição
```

```
"""
```

```
sumarizador(entrada, max_length=100, max_new_tokens=100)
```

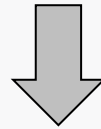
Fonte: https://huggingface.co/docs/transformers/model_doc/t5

Caso 6: Sumarização (2/2)

- **Resultado**

- O resumo não informou que o quati roubou a comida !!!

Uma turista que visitava o Cristo Redentor neste domingo (25) foi surpreendida por um quati que roubou o seu sanduíche e fugiu. O animal chegou devagarzinho e, sorrateiramente, meteu a patinha no lanche da moça e correu. Pessoas que estavam ao redor registraram a cena. Como o animal estava pertinho deles, os visitantes começaram a filmar e flagraram o animalzinho garantindo a refeição.



```
[{'summary_text': 'turista que visitava o Cristo Redentor neste domingo (25) foi surpreendida por um quati que chegou devagarzinho . pessoas que estavam ao redor registraram a cena .'}]
```

Caso 7: NER

- **Reconhecimento de Entidades Nomeadas (*Named Entity Recognition – NER*)**
 - Consiste em encontrar e identificar entidades mencionadas em um texto (como pessoas, locais, etc.)

```
ner_model = pipeline('ner', model='Babelscape/wikineural-multilingual-ner')
texto = """Cristiano Ronaldo nasceu em Portugal em 1985.
Ele chegou no Real Madrid no dia 6 de Julho de 2009
"""
ner_model(texto)
```

```
[{'entity': 'B-PER', 'score': np.float32(0.9998411), 'index': 1, 'word': 'Cristiano',
'start': 0, 'end': 9},
{'entity': 'I-PER', 'score': np.float32(0.9998635), 'index': 2, 'word': 'Ronaldo',
'start': 10, 'end': 17},
{'entity': 'B-LOC', 'score': np.float32(0.9993074), 'index': 5, 'word': 'Portugal',
'start': 28, 'end': 36},
{'entity': 'B-ORG', 'score': np.float32(0.99833256), 'index': 12, 'word': 'Real',
'start': 61, 'end': 65},
{'entity': 'I-ORG', 'score': np.float32(0.998968), 'index': 13, 'word': 'Madrid',
'start': 66, 'end': 72}]
```

Comentários Finais (1/2)

- Algumas Estatísticas do HuggingFace
 - + de 395.000 bases de dados em 8 mil idiomas
 - (*O número de linguagens naturais no planeta é menor que esse*)
 - + de 1.710.000 modelos,
 - 7.422 suportam Português,
 - + de 93.000 servem para classificação



Comentários Finais (2/2)

- **Método Pipeline**

- Facilita muito a utilização do Hugging Face
- Entretanto:
 - Apenas alguns modelos da plataforma podem ser utilizados no modo Pipeline
 - Como não é possível fazer configurações ou ajustes, o desempenho pode não ser tão bom
 - Tanto na questão da eficiência (velocidade) como na eficácia (qualidade dos resultados)
 - Você tem que escolher um modelo adequado ao seu problema, seus dados e idioma.
 - Idealmente, deve-se procurar conhecer um pouco sobre os conceitos que fundamentam o modelo escolhido.
 - Deve-se observar também a quantidade de parâmetros

Referências

- ALAMMAR, J. **The Illustrated Transformer**. Github.io, 2018. Disponível em: < <https://jalammar.github.io/illustrated-word2vec> >. Acesso em: 13 fev. 2025.
- ALURA. **Hugging face: o que é e como usar essa plataforma**. Alura, 2024. Disponível em: < <https://www.alura.com.br/artigos/hugging-face> >. Acesso em: 19 mai. 2025.
- CARRASCOSA, I. P. **Using Hugging Face Transformers for Emotion Detection in Text**. KDNuggets, 2024. Disponível em: < <https://www.kdnuggets.com/using-hugging-face-transformers-for-emotion-detection-in-text> >. Acesso em: 19 mai. 2025.
- CHIU, K.-L., ALEXANDER, R. **Detecting Hate Speech with GPT-3**. *In*: arXiv:2013:12407v1. 2021.
- JURAFSKY, D.; MARTIN, J. H. **The Transformer**. *In*: JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing**. 3. ed. (Jan. 12, 2025 draft). [s.l.]: [s.n.], 2025. cap 9. Disponível em: < <https://web.stanford.edu/~jurafsky/slp3/> >. Acesso em: 22 abr. 2025.
- HUGGING FACE. **Transformers library**. Hugging Face, 2025. Disponível em: < <https://huggingface.co/docs/transformers/en/index> >. Acesso em: 19 mai. 2025.